Problem 2.0.23

ee25btech11023-Venkata Sai

March 26, 2026

1. Problem
2. Solution
   - Representation
   - Tangency Condition
   - Calculation
3. Plot
4. Python Code

Problem

The maximum value of the function $2x + 3y + 4z$ on the ellipsoid $2x^2 + 3y^2 + 4z^2 = 1$ is:

Representation

The equation of the conic is given by

$$\mathbf{x}^\top \mathbf{V}\mathbf{x} + 2\mathbf{u}^\top \mathbf{x} + f = 0 \tag{1.1}$$

On comparing the given ellipsoid $2x^2 + 3y^2 + 4z^2 = 1$, we get

$$\mathbf{V} = \begin{pmatrix} 2 & 0 & 0 \\ 0 & 3 & 0 \\ 0 & 0 & 4 \end{pmatrix}, \mathbf{u} = \begin{pmatrix} 0 \\ 0 \\ 0 \end{pmatrix} \text{ and } f = -1 \tag{1.2}$$

The function $2x + 3y + 4z = \mathrm{c}$ is a plane with:

$$\mathbf{n}^\top \mathbf{x} = c \tag{1.3}$$

where

$$\mathbf{n} = \begin{pmatrix} 2 \\ 3 \\ 4 \end{pmatrix} \tag{1.4}$$

Tangency Condition

If $\mathbf{V}^{-1}$ exists, given the normal vector $\mathbf{n}$, the tangent points of contact to conic are given by

$$\mathbf{q} = \mathbf{V}^{-1}(\kappa_i \mathbf{n} - \mathbf{u}) \tag{1.5}$$

where

$$\kappa_i = \pm\sqrt{\frac{f_0}{\mathbf{n}^\top \mathbf{V}^{-1}\mathbf{n}}} \text{ and } f_0 = \mathbf{u}^\top \mathbf{V}^{-1}\mathbf{u} - f \tag{1.6}$$

Calculation

By substituting $\mathbf{q}$ in (1.3)

$$c = \mathbf{n}^\top \mathbf{V}^{-1}(\kappa\mathbf{n} - \mathbf{u}) = \pm \frac{f_o}{\sqrt{\mathbf{n}^\top \mathbf{V}^{-1}\mathbf{n}}} \left(\mathbf{n}^\top \mathbf{V}^{-1}\mathbf{n}\right) - \mathbf{n}^\top \mathbf{V}^{-1}\mathbf{u} \tag{1.7}$$

$$= \pm \sqrt{f_0(\mathbf{n}^\top \mathbf{V}^{-1}\mathbf{n})} - \mathbf{n}^\top \mathbf{V}^{-1}\mathbf{u} \tag{1.8}$$

$$= \pm \sqrt{\left(\mathbf{u}^\top \mathbf{V}^{-1}\mathbf{u} - f\right)\left(\mathbf{n}^\top \mathbf{V}^{-1}\mathbf{n}\right)} - \mathbf{n}^\top \mathbf{V}^{-1}\mathbf{u} \tag{1.9}$$

Given

$$\mathbf{V} = \begin{pmatrix} 2 & 0 & 0 \\ 0 & 3 & 0 \\ 0 & 0 & 4 \end{pmatrix} \implies \mathbf{V}^{-1} = \begin{pmatrix} \frac{1}{2} & 0 & 0 \\ 0 & \frac{1}{3} & 0 \\ 0 & 0 & \frac{1}{4} \end{pmatrix} \tag{1.10}$$

Substituting given values from (1.2)
As $\mathbf{u} = \mathbf{0}$,

$$\mathbf{u}^\top \mathbf{V}^{-1}\mathbf{u} = 0 \text{ and } \mathbf{n}^\top \mathbf{V}^{-1}\mathbf{u} = 0 \tag{1.11}$$

Calculation

$$\mathbf{n}^\top \mathbf{V}^{-1}\mathbf{n} = \begin{pmatrix} 2 & 3 & 4 \end{pmatrix} \begin{pmatrix} \frac{1}{2} & 0 & 0 \\ 0 & \frac{1}{3} & 0 \\ 0 & 0 & \frac{1}{4} \end{pmatrix} \begin{pmatrix} 2 \\ 3 \\ 4 \end{pmatrix} \tag{1.12}$$

$$= \begin{pmatrix} 1 & 1 & 1 \end{pmatrix} \begin{pmatrix} 2 \\ 3 \\ 4 \end{pmatrix} \tag{1.13}$$

$$= (1 \cdot 2) + (1 \cdot 3) + (1 \cdot 4) = 9 \tag{1.14}$$

Substituting these values in (1.9), we get

$$c = \pm\sqrt{9} - 0 \tag{1.15}$$

$$c = \pm\sqrt{9} = \pm 3 \tag{1.16}$$

Hence the maximum value of c is 3.

Finding the point

From (1.5),

$$\mathbf{q} = \mathbf{V}^{-1}(\kappa_i \mathbf{n} - \mathbf{u}) \tag{1.17}$$

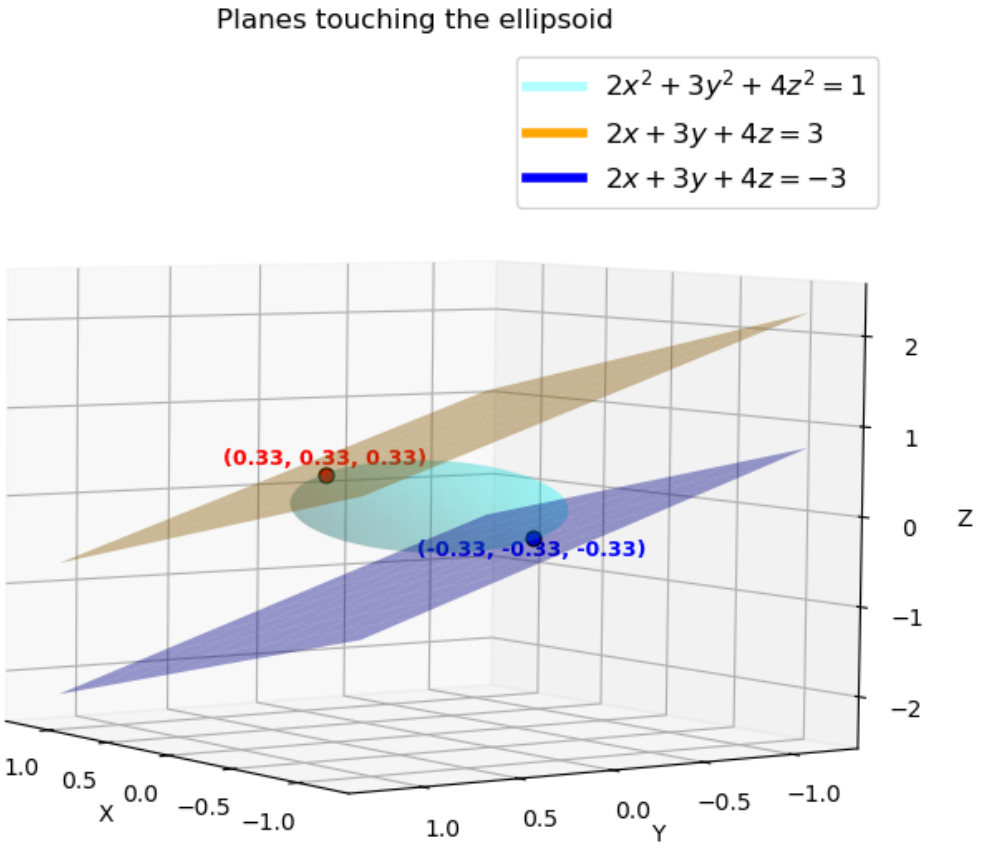
$$f_o = \mathbf{u}^\top \mathbf{V}^{-1}\mathbf{u} - f = 0 - (-1) = 1 \tag{1.18}$$

$$\kappa_i = \pm\sqrt{\frac{f_0}{\mathbf{n}^\top \mathbf{V}^{-1}\mathbf{n}}} = \pm\sqrt{\frac{1}{9}} = \pm\frac{1}{3} \tag{1.19}$$

$$\mathbf{q} = \begin{pmatrix} \frac{1}{2} & 0 & 0 \\ 0 & \frac{1}{3} & 0 \\ 0 & 0 & \frac{1}{4} \end{pmatrix} \left(\pm\frac{1}{3}\begin{pmatrix} 2 \\ 3 \\ 4 \end{pmatrix} - \mathbf{0} \right) \tag{1.20}$$

$$= \pm\frac{1}{3}\begin{pmatrix} \frac{1}{2} & 0 & 0 \\ 0 & \frac{1}{3} & 0 \\ 0 & 0 & \frac{1}{4} \end{pmatrix} \begin{pmatrix} 2 \\ 3 \\ 4 \end{pmatrix} = \begin{pmatrix} \pm\frac{1}{3} \\ \pm\frac{1}{3} \\ \pm\frac{1}{3} \end{pmatrix} \tag{1.21}$$

Plot


Planes touching the ellipsoid
$2x^2 + 3y^2 + 4z^2 = 1$
$2x + 3y + 4z = 3$
$2x + 3y + 4z = -3$
(0.33, 0.33, 0.33)
(-0.33, -0.33, -0.33)
X
Y
Z

Python Code for Plotting

```
import cvxpy as cp
import numpy as np
import matplotlib.pyplot as plt
from matplotlib.lines import Line2D

# Define the variables
x_var = cp.Variable(3)

# Define Matrix V and vector n
V_mat = np.diag([2, 3, 4])
n_vec = np.array([2, 3, 4])

# Maximize n^T x subject to x^T V x <= 1
objective = cp.Maximize(n_vec @ x_var)
constraints = [cp.quad_form(x_var, V_mat) <= 1]
prob = cp.Problem(objective, constraints)
```

Python Code for Plotting

```
# Solve the problem
prob.solve()

# Results
c_max = prob.value
q_max = x_var.value
q_min = -q_max # Symmetry of the ellipsoid

print(fMax value found: {c_max:.4f})
print(fPoint of maximization: {q_max})

# 2. Visualization setup
fig = plt.figure(figsize=(12, 8))
ax = fig.add_subplot(111, projection='3d')

# Generate Ellipsoid Data
u = np.linspace(0, 2 * np.pi, 100)
v = np.linspace(0, np.pi, 100)
```

Python Code for Plotting

```
rx, ry, rz = 1/np.sqrt(2), 1/np.sqrt(3), 1/np.sqrt(4)
x_ell = rx * np.outer(np.cos(u), np.sin(v))
y_ell = ry * np.outer(np.sin(u), np.sin(v))
z_ell = rz * np.outer(np.ones(np.size(u)), np.cos(v))

# Generate Tangent Planes
grid = np.linspace(-1.2, 1.2, 10)
X, Y = np.meshgrid(grid, grid)
Z_max = (c_max - n_vec[0]*X - n_vec[1]*Y) / n_vec[2]
Z_min = (-c_max - n_vec[0]*X - n_vec[1]*Y) / n_vec[2]

# Plot Ellipsoid
ax.plot_surface(x_ell, y_ell, z_ell, color='cyan', alpha=0.2,
    linewidth=0)

# Plot Planes
ax.plot_surface(X, Y, Z_max, color='orange', alpha=0.4)
ax.plot_surface(X, Y, Z_min, color='blue', alpha=0.4)
```

Python Code for Plotting

```
# Mark the points with reduced size
ax.scatter(q_max[0], q_max[1], q_max[2], color='red', s=40,
    edgecolors='k')
ax.scatter(q_min[0], q_min[1], q_min[2], color='blue', s=40,
    edgecolors='k')

# Label coordinates at the dots
ax.text(q_max[0], q_max[1], q_max[2]+0.1,
       f({q_max[0]:.2f}, {q_max[1]:.2f}, {q_max[2]:.2f}),
       color='red', fontsize=9, fontweight='bold', ha='center')
ax.text(q_min[0], q_min[1], q_min[2]-0.2,
       f({q_min[0]:.2f}, {q_min[1]:.2f}, {q_min[2]:.2f}),
       color='blue', fontsize=9, fontweight='bold', ha='center')

# Set view to between the planes
ax.view_init(elev=5, azim=145)
```

Python Code for Plotting

```
# Legend
legend_elements = [
    Line2D([0], [0], color='cyan', lw=4, alpha=0.3, label=r'$2x^2
         + 3y^2 + 4z^2 = 1$'),
    Line2D([0], [0], color='orange', lw=4, label=r'$2x + 3y + 4z
        = 3$'),
    Line2D([0], [0], color='blue', lw=4, label=r'$2x + 3y + 4z =
        -3$')
]
ax.legend(handles=legend_elements, loc='upper right', fontsize
    =12)

ax.set_xlabel('X')
ax.set_ylabel('Y')
ax.set_zlabel('Z')
ax.set_title('Planes touching the ellipsoid')

plt.show()
```